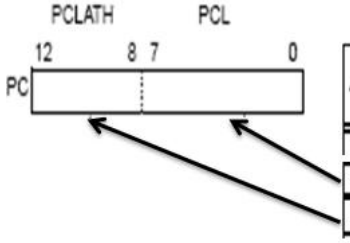
# TD : Jeux D'instructions

## Exercice 1 : PC/PCL

1. Quel est le rôle du compteur ordinal (dit Program Counter : PC)

**Le compteur ordinal contient l'adresse de la prochaine instruction à exécuter**

2. La taille du compteur ordinal (dit Program Counter PC) est 13 bits, sa valeur est lue dans les deux registres spéciaux PCLATH et PCL, comme suit :




| Address | Name | Bit 7 | Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0 | Value on Power-on Reset | Value on all other resets (Note3) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Bank 0 | | | | | | | | | | | |
| 02h | PCL | Low order 8 bits of the Program Counter (PC) | | | | | | | | 0000 0000 | 0000 0000 |
| 0Ah | PCLATH | — | — | — | Write buffer for upper 5 bits of the PC (1) | | | | | ---0 0000 | ---0 0000 |

Sachant qu'avec le PIC 16F84, le nombre d'instructions maximal tolérable est de 1024, quelles sont les valeurs possibles du registre PCLATH? Expliquer.

**Les valeurs possibles sont quatre : 00, 01, 10 et 11 Pour notre PIC16F84, la longueur d'adresse d'une instruction est de 10 bits : le PCL (PClow) contient les 8 bits de poids faible et le PCLATH contient les 2 bits de poids fort**

3. Commentez et analysez le bout de code suivant et dire c'est quoi le résultat final des registres W,PCL et resultat (resultat est une variable déjà déclarée) :

| | | |
|---|---|---|
| Movlw 0x02 | | ; W=2 |
| Call tab | | ; appel de la sous-routine « tab » |
| Movwf resultat | | ;Resultat = w = 0x0C |
| Tab | ; nom de la sous-routine | |
| | Addwf PCL,f | ;PCL = PCL+w = l'adresse de la prochaine instruction + 2 |
| | Retlw 0x0A | ; |
| | Retlw 0x0B | ;…… …… |
| | Retlw 0x0C | ; Retour avec W =0X0C |
| | Retlw 0x0D | …… |

Rappel : L'instruction « retlw » assure le retour d'une sous-routine (équivaut à l'instruction return), mais en plus, elle affecte à W une valeur spécifiée :
Syntaxe : retlw k ; (w) = k puis return.

## Exercice 2 : Jeux d'instructions

Quel est le contenu de var et de w après l'exécution de ces instructions ? et précisez l'état des bits modifiés au niveau du registre STATUS : Z et C

Sachant qu'on a :
CBLOCK 0x0E
Var :1
endC

Compléter le tableau ci-après

| Instruction | Description | VAR | W | C | Z |
|---|---|---|---|---|---|
| movlw 0x05<br>movwf var | W =5<br>Var =w =5 | 5 | 5 | | |
| BCF var,2 | Var.2=0 | 1 | 5 | | |
| incf var,1 | Var=var + 1 | 2 | 5 | | 0 |
| decf var, w | W=var-1 | 2 | 1 | | 0 |
| movlw 255<br>addlw 1 | W=255<br>W=255+1=0 | 2 | 0 | 1 | 1 |
| movlw 12<br>movwf var<br>movlw 25<br>addwf var,f | W=12 | 2 | 12 | 1 | 1 |
| | Var=w=12 | 12 | 12 | 1 | 1 |
| | W=25 | 12 | 25 | 1 | 1 |
| | Var=var+w | 37 | 25 | 0 | 0 |
| movlw 0x02<br>sublw 0x01 | W=2<br>W=2-1=1 | 37 | 1 | 0 | 0 |
| movlw 0x20<br>movwf var<br>movlw 0x1F<br>subwf var,w<br>movwf var | W=20h | 37 | 0010 0000 | 0 | 0 |
| | Var=w | 0010 0000 | 0010 000 | 0 | 0 |
| | W=1Fh | 0010 0000 | 0001 1111 | 0 | 0 |
| | W=20h-1Fh | 0010 0000 | 0000 0001 | 0 | 0 |
| | Var=w | 0000 0001 | 0000 0001 | 0 | 0 |
| movlw B'11001101'<br>andlw B'11110000' | W=1100 1101<br>W=1100 1101&& 11110000 =1100 0000 | | 1100 1101<br>1100 0000 | 0<br>0 | 0<br>0 |
| movlw B'11000101'<br>xorlw B'00001111' | W=11000101<br>W=11000101 xor 00001111 = 11001010 | | 11000101<br>11001010 | 0<br>0 | 0<br>0 |
| BCF status,c<br>Movlw B'10010111'<br>Movwf var | Status.0=0<br>W=10010111<br>Var=w | <br><br>10010111 | <br>10010111 | | 0 |
| Movlw 0xC5<br>Movwf var | W = C5h<br>Var=w | <br>11000101 | 1100 0101 | | |

## Exercice 3 : Comparaison

Soient trois variables A , B et resultat, écrire le code assembleur commenté qui permet de les comparer :
- Si A>B, resultat =1
- Si A=B, resultat=2
- Si A<B, resulktat=3

1- Déclarer les variables suivantes dans l'ordre (resultat, A, B) chacune de taille 1 octet

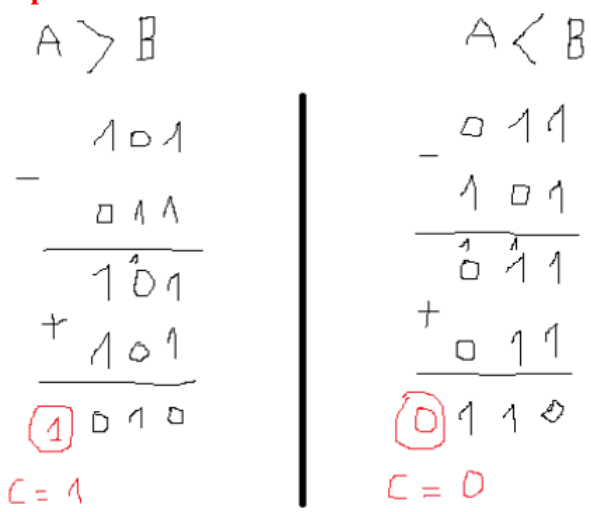
```
CBLOCK 0X0C
        Resultat :1     ; 0x0C
        A : 1           ; 0x0D
        B : 1           ; 0x0E
endC
```

Quelle est l'adresse de la variable B au niveau de la RAM ?

**0X0E**

2- Code :

**Exemple : 5-3 & 3-5**




**5-3=101-011=101+101=1010 (AsupB) → c=1**

**3-5=011-101=011+011=0110 (AinfB) → c=0**

```
Compare  mov B,w
         Subwf A,w
         BTFSC status,z
         Goto affiche2
         BTFSS status,c
         Goto affiche1
         Goto affiche3

Affiche1  movlw 1
          Movwf resultat
          Goto compare

Affiche2 movlw 2
         Movwf resultat
         Goto compare

Affiche3 movlw 3
         Movwf resultat
         Goto compare
```

Application : MPlab + ISIS

On se propose d'implémenter sur un PIC16F84 un comparateur ayant 1 entrée de 2 bits . Le fonctionnement est le suivant :

-Si l'entrée (A1,A0)2 est égale à 2 alors la diode verte s'allume et l'afficheur 7 Segment affiche E

-Si l'entrée (A1,A0)2 est inférieure à 2 alors la diode rouge s'allume et l'afficheur 7 Segment affiche I

-Si l'entrée (A1,A0)2 est supérieure à 2 alors la diode bleue s'allume et l'afficheur 7 Segment affiche S

1) Donnez l'organigramme
2) Proposer un code en assembleur répondant au fonctionnement décrit
3) Réaliser le câblage correspondant (figure 1) sur ISIS et simuler le fonctionnement

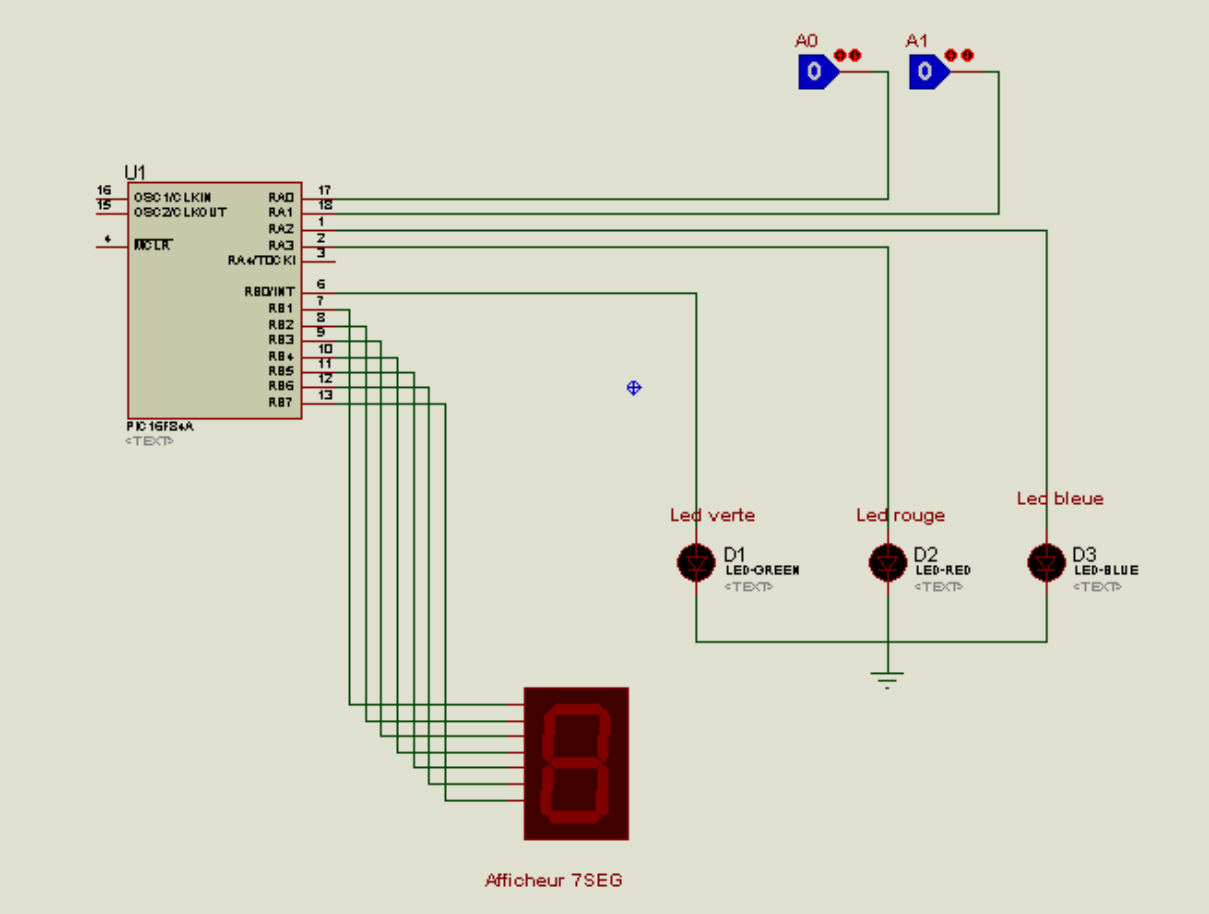
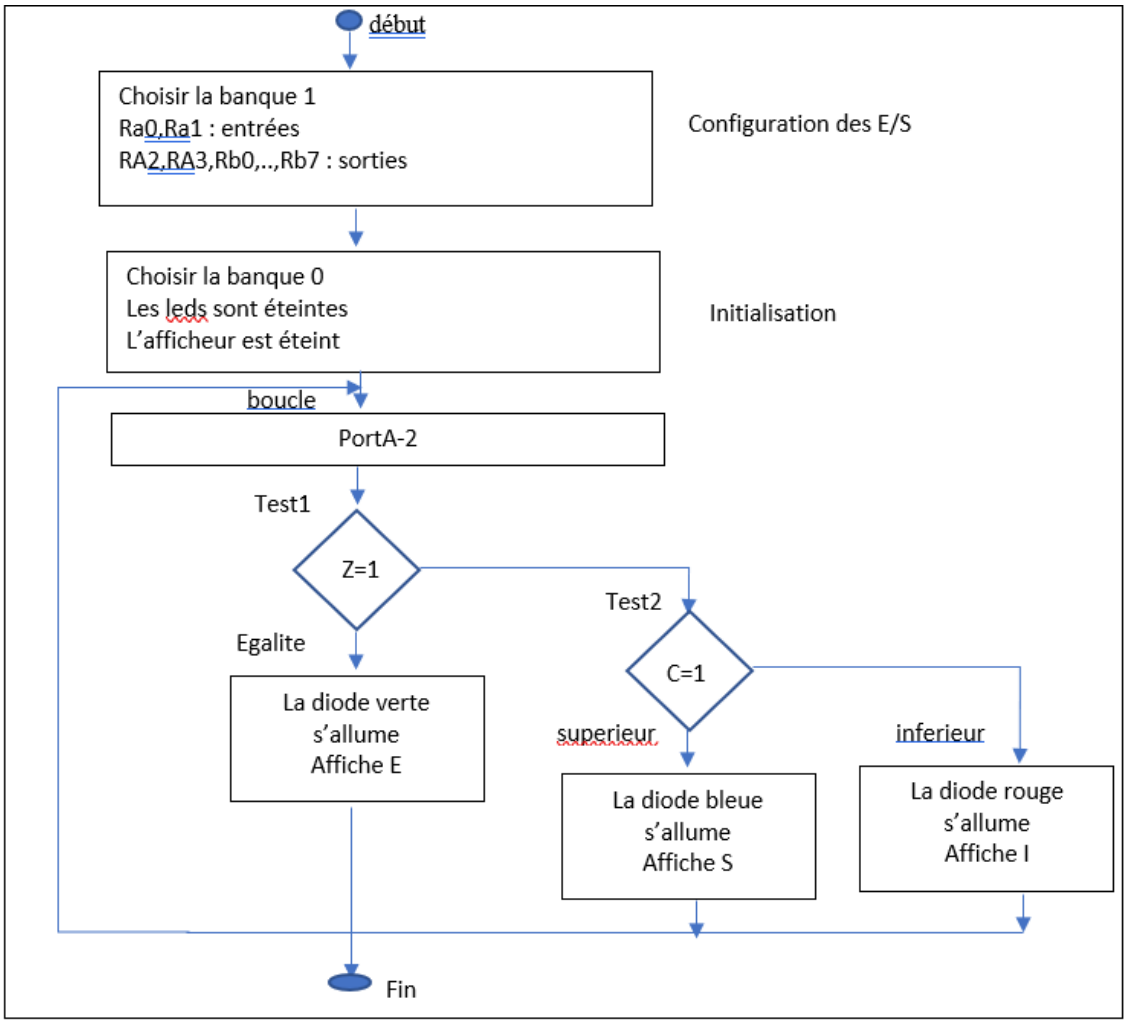



Figure 1

1-




2-

```
LIST      p=16F84          ; Définition de processeur
 #include <p16F84.inc>       ; Définitions de variables

 __CONFIG   _CP_ON & _WDT_OFF & _PWRTE_ON & _HS_OSC

#define L_A1A0_egal_2 portb,0
#define L_A1A0_sup_2 porta,2
#define L_A1A0_inf_2 porta,3

org 0x0005
;config
BSF status, rp0
movlw 0x03
movwf trisa
clrf trisb
```

```
;initialisation
BCF status,rp0
clrf portb
BCF L_A1A0_sup_2
BCF L_A1A0_inf_2

compare
  movlw 0x03
  andwf porta,w; masque de deux premiers bits du porta
  sublw 2 ; w=2-Porta

BTFSc status,z
goto egalite ;F (z=1)
BTFSS STATUS,C ;V (z=0)
goto A_sup_2
goto A_inf_2

egalite
BCF L_A1A0_sup_2
BCF L_A1A0_inf_2
Movlw b'11110011' ; 0XF3
movwf portb
goto compare

A_sup_2
BSF L_A1A0_sup_2
BCF L_A1A0_inf_2
Movlw b'11011010' ; 0XDA
movwf portb
goto compare

A_inf_2
BSF L_A1A0_inf_2
BCF L_A1A0_sup_2
Movlw b'00001100'  ; 0X 0C
movwf portb
goto compare

END
```